

See discussions, stats, and author profiles for this publication at: <https://www.researchgate.net/publication/393498778>

APPLICATION OF RETRIEVAL-AUGMENTED GENERATION (RAG) IN CHATBOT FOR COSMETICS Q&A

Conference Paper · July 2025

CITATIONS

0

READS

59

4 authors, including:



[Le Duy Tan](#)

Ho Chi Minh City International University

50 PUBLICATIONS 199 CITATIONS

[SEE PROFILE](#)

APPLICATION OF RETRIEVAL-AUGMENTED GENERATION (RAG) IN CHATBOT FOR COSMETICS Q&A

Vo Ngoc Minh Anh^{1,3*}, Pham Le Duc Thinh^{2,3}, Tan Duy Le^{2,3},
Le Anh Tien^{4*}, Le Dieu Linh⁵

¹*University of Science, Vietnam*

²*School of Computer Science and Engineering, International University, Vietnam*

³*Vietnam National University, Vietnam*

⁴*FPT University, Vietnam*

⁵*Hong Duc University, Vietnam*

**Email: tienla6@fe.edu.vn, Phone: 0913176383*

ABSTRACT

The increasing demand for skincare has highlighted the importance of selecting suitable cosmetics, thereby driving the development of chatbot systems designed to assist in cosmetic consultations. Although Retrieval-Augmented Generation (RAG) technology has been extensively employed across domains such as healthcare, education, and e-commerce, its application within the cosmetics sector remains relatively underexplored. In this study, we propose a RAG-based chatbot system that integrates information retrieval and text generation techniques to generate accurate, contextually appropriate responses tailored to users' specific needs. The system utilizes structured and organized cosmetic product data stored in MongoDB, with vectorization management handled by Milvus and query optimization supported by the multilingual-e5-large model. Furthermore, the chatbot integrates the GPT-4 model with Function Calling capabilities, enabling it to perform tasks and deliver precise responses based on user queries. Deployed on the Messenger platform, the system leverages FastAPI and Nginx to ensure both performance efficiency and robust security. Experimental results demonstrate that the proposed system significantly outperforms traditional search methods, offering faster and more accurate responses. This research opens new avenues for developing personalized chatbot solutions that enhance the efficacy of cosmetic consultations and assist users in selecting the most suitable products for their individual needs.

Keywords: *retrieval-augmented generation, large language model, cosmetics recommendation chatbot, function calling, semantic search.*

1. INTRODUCTION

In recent years, the cosmetics industry has seen increased demand for personalized beauty care, with consumers seeking tailored shopping experiences and expert consultations [1]. The rise of artificial intelligence (AI), particularly intelligent chatbots, has improved customer service by enhancing efficiency and personalization. Central to this is natural language processing (NLP), which enhances communication and contextual understanding, playing a key role in improving the customer experience in cosmetics [2].

Despite the widespread adoption of AI chatbots, their effectiveness in cosmetics remains limited by challenges in retrieving information from unstructured sources such as product descriptions, ingredient lists, user reviews, and market trends [3]. Traditional rule-based and statistical methods struggle with complex queries and rapid adaptation to new data. In contrast, deep learning models like GPT and BERT provide richer semantic representations and superior information extraction [4]. Retrieval-Augmented Generation (RAG) combines the generative capabilities of large language models (LLMs) with external retrieval to ground responses in up-to-date, domain-specific data [5]. This research proposes integrating RAG with MongoDB and Milvus to optimize retrieval from heterogeneous databases.

The goal of this study is to develop an intelligent chatbot that leverages RAG for personalized cosmetic consultations. Specifically, we focus on:

Design and implementation of a hybrid RAG pipeline combining structured filtering in MongoDB with vector-based ranking in Milvus; In-depth multilingual evaluation on Vietnamese-language queries using the multilingual-e5-large embedding model, reporting Precision, Recall, MRR and MAP; Production-ready deployment on Messenger via FastAPI and Nginx demonstrates real-world feasibility of GPT-4 function calling.

2. BACKGROUND AND RELATED WORK

2.1. Background

In recent years, the rapid advancement of science and technology, particularly in the field of LLMs, has significantly enhanced the capabilities of chatbots. These developments have transformed chatbots into highly effective tools across various domains, especially in customer support. Traditionally, customer service relied heavily on phone calls, and community forums. However, beginning in the 2010s, chatbots began to emerge as the primary medium for customer interaction, offering faster response times and improved accessibility [6].

Automating simple tasks allows human agents to address more complex issues, improving service efficiency [7]. Chatbots are typically classified into two types: task-oriented systems, which handle specific functions such as booking or FAQs, and conversational systems, which simulate human dialogue [8].

To improve performance, fine-tuning is often applied to LLMs post pre-training, enhancing emotional intelligence and domain alignment [9]. RAG is another method that integrates LLMs with external knowledge sources, improving the factual accuracy of responses [10, 11].

Despite improvements, challenges remain in storing and retrieving knowledge effectively, particularly with unstructured or semi-structured data. Vector databases such as Milvus, Redis, and MongoDB address this by storing high-dimensional embeddings and enabling efficient semantic search [12]. The performance of chatbot systems strongly depends on the selection of storage architecture and retrieval algorithms.

This study proposes combining MongoDB for structured data with Milvus for vector-based semantic search to enhance response accuracy and retrieval speed, particularly in complex query scenarios.

2.2. Related Work

Recent studies have explored ways to enhance LLM-based chatbots through fine-tuning and retrieval integration. Fine-tuning has been shown to improve emotional and contextual alignment in dialogue systems [9]. In the financial domain, Yepes et al. [10] proposed a chunking method to improve the effectiveness of retrieval-augmented generation, enabling more accurate responses from lengthy reports.

Vector embeddings have shown strong performance in semantic search, enabling more accurate query matching. Their implementation in vector databases such as Milvus has been shown to support real-time, large-scale applications [12]. In contrast, structured databases like MongoDB are efficient for organized data storage but do not support semantic similarity search.

Hybrid approaches that integrate structured storage with semantic retrieval have been proposed but often lack cohesion in architecture. This study addresses that gap by presenting a unified system that leverages both MongoDB and Milvus to support real-time, semantically enriched chatbot interactions.

3. MATERIALS AND METHODS

3.1. Data Collection and Preprocessing

The dataset was sourced from Hasaki [13], an e-commerce platform specializing in cosmetics. It includes key attributes such as product category, name, discounted price, original price, and image links, along with descriptions, ingredients, usage instructions, and specifications. The dataset also incorporated data on skin type compatibility, user ratings and reviews, brand information, and stock availability. It includes 7,315 products from 415 brands across 113 categories, providing a robust foundation for analysis.

To ensure data quality, preprocessing was applied, including:

Noise removal: Eliminating irrelevant characters (e.g., @, #, *, !, SKU codes, currency symbols).

Standardization: Normalizing numerical fields (e.g., converting “1.500.000 VND” to “1500000”).

Data restructuring: Splitting merged fields, removing redundant information, and creating new fields.

3.2. System Architecture and Database Design

The system was designed using RAG for intelligent querying and personalized recommendations. The storage and retrieval infrastructure consists of MongoDB (structured data) and Milvus (vectorized data).

The architecture comprises several key components, including data storage, query processing, and chatbot integration. MongoDB was chosen for its flexibility in handling structured and semi-structured data, with indexed fields such as category, price, and brand for optimized query performance. Milvus was integrated to store vector embeddings of

product descriptions, ingredients, and user reviews, using the multilingual-e5-large model for multilingual natural language processing [14]. We evaluated several multilingual embedding models including multilingual-e5-large, LaBSE, mUSE, and GTR-XLarge on Vietnamese product search benchmarks. According to the Massive Text Embedding Benchmark (MTEB) and Multilingual Information Retrieval Across a Continuum of Languages (MIRACL) benchmarks, multilingual-e5-large outperforms others with higher Mean Reciprocal Rank (MRR) and normalized Discounted Cumulative Gain (nDCG) scores on Vietnamese queries, due to its large-scale multilingual training and 1024-dimensional embeddings that enhance semantic discrimination in e-commerce contexts [14, 15, 16]. Older models like LaBSE and mUSE show lower retrieval effectiveness on Vietnamese data. Therefore, the multilingual-e5-large model demonstrates robust zero-shot performance on Vietnamese semantic search tasks, eliminating the need for fine-tuning, making it an optimal choice for our system. Milvus is an open-source vector database designed for efficient storage and retrieval of high-dimensional embeddings using Approximate Nearest Neighbor (ANN) algorithms such as Inverted File Index (IVF) and Hierarchical Navigable Small World (HNSW), achieving sub-second query latency even at billion-scale vector datasets [17]. It supports horizontal scaling and integrates well with AI pipelines, making it suitable for semantic search in real-time applications. MongoDB is a flexible NoSQL document store that efficiently manages structured and semi-structured metadata with rich indexing capabilities for filtering and querying [18]. The hybrid approach of first filtering metadata in MongoDB, followed by semantic vector search in Milvus, effectively combines rapid attribute-based filtering with high-accuracy semantic retrieval. This method has been demonstrated to perform well in industry-scale systems [19]. Figure 1 illustrates the complete system architecture, showing the flow from raw data ingestion, document embedding, metadata storage, and vector-based search, to final interaction with the user through a large language model.

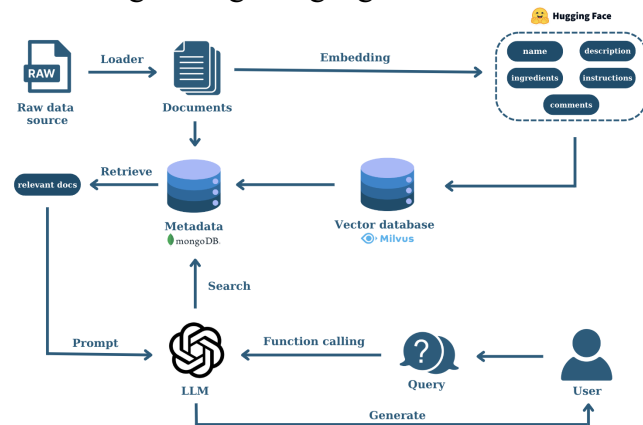


Figure 1. Overall architecture of the proposed RAG-based cosmetic-consultation system.

3.3. Query Processing and Ranking Mechanism

A function-calling mechanism based on GPT-4 was implemented to interpret user queries, extracting key attributes such as product type, budget constraints, and skin type preferences. The retrieval process follows a hybrid search strategy:

- **MongoDB Filtering:** Retrieves relevant products based on structured attributes like price range, brand, category, and skin type.
- **Milvus Vector Search:** Ranks filtered products based on semantic similarity to the user query across multiple fields.

The final similarity score $S_{avg}(p)$ for a product p is computed as the average of similarity scores across five distinct fields, as shown in Equation (1):

$$S_{avg}(p) = \frac{S_{ingredient}(p) + S_{instruction}(p) + S_{description}(p) + S_{comment}(p) + S_{name}(p)}{N} \quad (1)$$

Where:

$S_{ingredient}(p)$ is the similarity score based on product ingredients.

$S_{instruction}(p)$ is the similarity score based on usage instructions.

$S_{description}(p)$ is the similarity score based on product description.

$S_{ingredient}(p)$ is the similarity score based on product ingredients.

$S_{instruction}(p)$ is the similarity score based on usage instructions.

$S_{description}(p)$ is the similarity score based on product description.

As defined in Equation (1), this scoring method ensures that recommendations incorporate both structured product metadata and semantically rich textual signals, leading to more contextually relevant results.

3.4. API Development and Deployment

A FastAPI-based backend was developed to handle user queries and integrate chatbot functionalities. FastAPI was chosen for its high performance, asynchronous support, and built-in documentation generation. The system was containerized using Docker Compose, managing services like MongoDB, Milvus, and API servers. Deployment occurred on a cloud server with Nginx as a reverse proxy for load balancing.

3.5. Chatbot Integration

To enhance user accessibility, the chatbot was deployed on Facebook Messenger using the BotBanHang platform. This allows seamless interaction, real-time recommendations, and improved user engagement.

3.6. Evaluation Metrics

Precision: Evaluates the accuracy of the system by measuring the proportion of relevant results among all retrieved results.

Context Precision: Assesses how well the system's results align with the specific context and intent of the user's query.

Recall: Measures the system's ability to retrieve relevant results, reflecting the proportion of relevant results successfully identified.

Relevance: Examines how well the system's results satisfy the user's information needs and match the intent of the query.

Entities Recall: Evaluates the system’s ability to correctly identify and retrieve relevant entities.

Time: Measures the system’s efficiency by evaluating the time required to retrieve and present results after receiving a query.

4. RESULTS

The research used a dataset of 7,315 products from 415 brands and 113 categories, collected from various e-commerce websites. Data preprocessing included removing redundant characters, SKUs, special symbols, and standardizing prices. We introduced a new field, “volume_weight”, which converts either the product’s net weight (grams) or volume (millilitres) into a single numeric attribute for downstream filtering and comparison. The cleaned data was stored in MongoDB for attribute queries and converted into 1024-dimensional vectors for semantic search in Milvus. A search system was developed by integrating GPT-4 with RAG and Function Calling techniques to extract key attributes like category, skin compatibility, brand, and price range. The system filters data in MongoDB, retrieves relevant product IDs, and uses Milvus with the IVF_FLAT index to assess semantic similarity and rank results based on similarity scores.

GPT-4 then converts these results into natural language responses. For example, given a Vietnamese-language query such as “kem nền Maybelline kiềm dầu dưới 300” (English: “Maybelline oil-control foundation under 300 VND”), the system first filters products by brand, category, oil-control properties, and price range in MongoDB, then applies a semantic search in Milvus, returning detailed information such as name, price, description, image, purchase link, and rating. Figure 2 illustrates the process, showing server interaction (left) and user response in the Messenger interface (right).

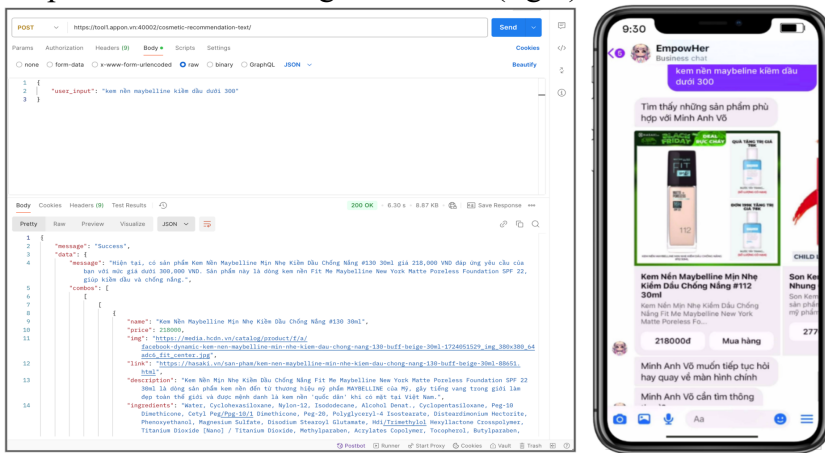


Figure 2. Server query interaction (left) and Messenger response (right).

Table 1. Evaluation Metrics for Proposed Method.

Metric	Precision	Context-Precision	Recall	Relevance	Entities Recall	Time (s)
Proposed Solution	0.95	0.89	1	0.91	0.95	7.8

The Hybrid method, which integrates MongoDB and Milvus, demonstrates superior performance in processing complex queries, particularly in applications that require high

precision and rapid response times. As presented in Table 1, the evaluation metrics for the Hybrid method exhibit remarkable results: Precision is 0.95, Context Precision is 0.89, Recall is 1.00, Entities Recall is 0.95, and Relevance is 0.91. These values indicate the method’s strong capabilities in providing precise, contextually relevant, and comprehensive search results. Furthermore, the average processing time for each query is only 7.8 seconds, highlighting the system’s high efficiency and fast response times.

Table 2. Comparison of MongoDB, Milvus, and Hybrid (MongoDB + Milvus) Methods across Various Search Factors.

Factors	MongoDB	Milvus	Hybrid Method (MongoDB + Milvus)
Filter-based Search	✓	✗	✓
Vector-based Search	✗	✓	✓
Semantic Search	✗	✓	✓
Search Speed	✓	✓	✓
Feedback Tuning Capability	✓	✗	✓
Handling Complex Data	✓	✓	✓
Handling Complex Queries	✓	✓	✓
Real-time Data Processing Capability	✓	✓	✓
Multilingual Support	✗	✓	✓

As illustrated in Table 2, the Hybrid method combines the advantages of both MongoDB and Milvus, enabling comprehensive search capabilities, including filter-based, vector-based, and semantic searches. The Hybrid method retains key strengths such as fast search speed, real-time data processing, and multilingual support. In contrast, standalone methods like MongoDB and Milvus typically offer only a subset of these features and are limited in their ability to handle complex queries or provide comprehensive search functionalities.

The findings show that the Hybrid approach excels in handling complex queries, offering potential for integrating LLMs and RAG methods. These advancements are particularly valuable in consulting and e-commerce, where precision, speed, and multilingual capabilities are crucial. The beauty industry, in particular, stands to benefit from accurate, fast, and contextually aware search solutions.

5. CONCLUSION & DISCUSSION

In this paper, we present the design and implementation of a cosmetic-consultation chatbot built upon a RAG framework. Our system combines structured filtering via MongoDB with semantic vector search powered by Milvus, and exploits GPT-4’s Function Calling API to parse user queries and orchestrate specialized subroutines. When evaluated on a corpus of 7,315 products covering 415 brands, the proposed hybrid approach achieved a precision of 0.95, a context precision of 0.89, a recall of 1.00 and a relevance score of 0.91, while maintaining an average response latency of just 7.8 seconds. Deployment on Facebook Messenger, facilitated by FastAPI and Nginx, demonstrates the solution’s real-

world viability in terms of both performance and security, thereby laying a robust foundation for a personalized chatbot capable of guiding users toward rapid and accurate product selections.

Despite these encouraging results, several limitations remain. First, our current product-classification mechanism relies on a fixed taxonomy that may prove insufficiently flexible as novel cosmetic categories or niche segments emerge. Second, the system's ability to scale under heightened query volumes and rapidly evolving product catalogs has yet to be rigorously assessed. Third, the prototype supports only text-based inputs; the incorporation of multimodal data such as product images or audio and video-based user reviews could further enrich interactions and improve recommendation accuracy.

Future work will pursue three main directions. We will develop a dynamic ontology module that leverages machine learning to autonomously update classification labels and expand the product hierarchy. We also plan to investigate distributed vector-search architectures to preserve low retrieval latencies at scale. Finally, we will extend the chatbot's input-processing capabilities to include real-time image recognition, speech-to-text conversion and sentiment analysis, thus creating a more flexible, scalable and user-centric system for e-commerce applications in the cosmetics domain.

6. ACKNOWLEDGEMENTS

We sincerely appreciate the AIoT Lab Vietnam for their indispensable support and contributions to this research.

7. REFERENCES

- [1] A. Eyeberdiyeva and Y. Gulberdiyeva, "Building a personalized cosmetics platform for enhanced customer experience", *Инновационная наука*, vol. 12, no. 2-1, pp. 39-41, 2024.
- [2] K. N. Vavliakis, G. Katsikopoulos, and A. L. Symeonidis, "E-commerce personalization with Elasticsearch", *Procedia Computer Science*, vol. 151, pp. 1128-1133, 2019.
- [3] J. Jnoub and D.-I. N. Nour, "Structured and unstructured textual data", 2022.
- [4] M. U. Hadi et al., "Large language models: A comprehensive survey of its applications, challenges, limitations, and future prospects", *Authorea Preprints*, vol. 1, pp. 1-26, 2023.
- [5] G. Sai and A. Purwar, "Context-augmented retrieval: A novel framework for fast information retrieval based response generation using large language model", *arXiv preprint arXiv:2406.16383*, 2024.
- [6] M. Nuruzzaman and O. K. Hussain, "Intellibot: A dialogue-based chatbot for the insurance industry", *Knowledge-Based Systems*, vol. 196, p. 105810, 2020.
- [7] L. Meyer-Waarden et al., "How service quality influences customer acceptance and usage of chatbots", *SMR-Journal of Service Management Research*, vol. 4, no. 1, pp. 35-51, 2020.
- [8] W. Brown, G. Wilson, and O. Johnson, "Understanding the role of chatbots in enhancing customer service", 2024.
- [9] K. Pandya and M. Holia, "Automating customer service using LangChain: Building custom open-source GPT chatbot for organizations", *arXiv preprint arXiv:2310.05421*, 2023.

- [10] A. J. Yepes, Y. You, J. Milczek, S. Laverde, and R. Li, “Financial report chunking for effective retrieval augmented generation”, arXiv preprint arXiv:2402.05131, 2024.
- [11] A. Masoudi, “AggroDoc-Semantic search using vector databases and large language models”, 2024.
- [12] M. Mickel, “Development and optimization of a retrieval augmented generation system for enhanced conversational AI assistance”, 2024.
- [13] Hasaki.vn, “Mua sắm mỹ phẩm chính hãng – Hasaki luôn bán đúng giá.” [Online]. Available: <https://hasaki.vn>. [Accessed: 27-May-2025].
- [14] L. Wang, N. Yang, X. Huang, L. Yang, R. Majumder, và F. Wei, “Multilingual e5 text embeddings: A technical report”, arXiv preprint arXiv:2402.05672, 2024.
- [15] N. Muennighoff, N. Tazi, L. Magne, và N. Reimers, “MTEB: Massive text embedding benchmark”, arXiv preprint arXiv:2210.07316, 2022.
- [16] X. Zhang, N. Thakur, O. Ogundepo, E. Kamalloo, D. Alfonso-Hermelo, X. Li, et al., “Miracl: A multilingual retrieval dataset covering 18 diverse languages”, Trans. Assoc. Comput. Linguist., vol. 11, pp. 1114–1131, 2023.
- [17] Milvus Documentation, “Overview”, Milvus, [Online]. Available: <https://milvus.io/docs/overview.md>. Accessed: May 29, 2025.
- [18] MongoDB Documentation, “Data Model”, MongoDB, [Online]. Available: <https://www.mongodb.com/docs/manual/core/document/>. Accessed: May 29, 2025.
- [19] Y. Li et al., “Hybrid Search Architecture Combining Vector and Metadata Filtering”, in Proc. ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, 2022.

ỨNG DỤNG MÔ HÌNH TẠO SINH TĂNG CƯỜNG TRUY XUẤT (RAG) TRONG CHATBOT HỎI ĐÁP CHUYÊN SÂU VỀ MỸ PHẨM TÓM TẮT

Nhu cầu chăm sóc da ngày càng tăng cao khiến việc chọn mỹ phẩm phù hợp quan trọng, thúc đẩy sự phát triển của chatbot tư vấn. Mặc dù công nghệ Retrieval-Augmented Generation (RAG) đã được áp dụng trong nhiều lĩnh vực như y tế, giáo dục, và thương mại điện tử, nhưng ứng dụng trong ngành mỹ phẩm vẫn còn hạn chế. Nghiên cứu này hướng đến xây dựng hệ thống chatbot ứng dụng RAG, kết hợp tìm kiếm thông tin và sinh văn bản để tạo phản hồi chính xác và phù hợp với nhu cầu của người dùng. Dữ liệu sản phẩm mỹ phẩm được thu thập, cấu trúc hóa và lưu trữ trong MongoDB, dùng Milvus quản lý vector hóa và tối ưu truy vấn bằng mô hình multilingual-e5-large. Ngoài ra, chatbot tích hợp mô hình ngôn ngữ lớn GPT-4 kết hợp Function Calling, giúp thực thi tác vụ và tạo phản hồi chính xác theo truy vấn của người dùng. Hệ thống được triển khai trên nền tảng Messenger, sử dụng FastAPI và Nginx để tối ưu hiệu suất và bảo mật. Kết quả

thử nghiệm cho thấy hệ thống vượt trội so với phương pháp tìm kiếm truyền thống, khả năng phản hồi nhanh và chính xác. Nghiên cứu này mở ra cơ hội phát triển chatbot cá nhân hóa, giúp nâng cao hiệu quả tư vấn mỹ phẩm, và hỗ trợ người dùng chọn sản phẩm phù hợp.

Từ khóa: truy xuất tăng cường tạo sinh, mô hình ngôn ngữ lớn, chatbot tư vấn mỹ phẩm, gọi hàm chức năng, tìm kiếm ngữ nghĩa.